

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Класс
6	seeds.zip	Последняя колонка

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from mpl_toolkits.mplot3d import Axes3D

df = pd.read_csv('seeds_dataset.txt', sep=r'\s+', header=None, na_values=[''])

df.fillna(df.mean(), inplace=True)

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Ручная реализация PCA (NumPy)
cov_matrix = np.cov(X_scaled, rowvar=False)

eigen_values, eigen_vectors = np.linalg.eig(cov_matrix)

eigen_values = eigen_values.real
eigen_vectors = eigen_vectors.real

sort_index = np.argsort(eigen_values)[::-1]
eigen_values = eigen_values[sort_index]
eigen_vectors = eigen_vectors[:, sort_index]

PC2_manual = np.dot(X_scaled, eigen_vectors[:, 0:2])
PC3_manual = np.dot(X_scaled, eigen_vectors[:, 0:3])

# Использование scikit-learn
pca2 = PCA(n_components=2)
PC2_sklearn = pca2.fit_transform(X_scaled)

pca3 = PCA(n_components=3)
PC3_sklearn = pca3.fit_transform(X_scaled)

full_info = eigen_values.sum()

reduce_info_2 = eigen_values[0:2].sum()
loss_2 = 100 - (reduce_info_2 / full_info * 100)

reduce_info_3 = eigen_values[0:3].sum()
loss_3 = 100 - (reduce_info_3 / full_info * 100)

print("--- Расчет потерь информативности ---")
```

```

print(f"Потери информативности при переходе к 2 компонентам: {loss_2:.2f}%")
print(f"Потери информативности при переходе к 3 компонентам: {loss_3:.2f}%\n")

classes = np.unique(y)
colors = ['red', 'green', 'blue']

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for i, cls in enumerate(classes):
    mask = (y == cls)
    axes[0].scatter(PC2_manual[mask, 0], PC2_manual[mask, 1],
                    c=colors[i], label=f'Class {int(cls)}', alpha=0.7)
axes[0].set_title('Метод 1: Ручной PCA (NumPy) - 2 компоненты')
axes[0].set_xlabel('Главная компонента 1 (PC1)')
axes[0].set_ylabel('Главная компонента 2 (PC2)')
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.5)

for i, cls in enumerate(classes):
    mask = (y == cls)
    axes[1].scatter(PC2_sklearn[mask, 0], PC2_sklearn[mask, 1],
                    c=colors[i], label=f'Class {int(cls)}', alpha=0.7)
axes[1].set_title('Метод 2: Scikit-Learn PCA - 2 компоненты')
axes[1].set_xlabel('Главная компонента 1 (PC1)')
axes[1].set_ylabel('Главная компонента 2 (PC2)')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

fig = plt.figure(figsize=(14, 6))

ax1 = fig.add_subplot(121, projection='3d')
for i, cls in enumerate(classes):
    mask = (y == cls)
    ax1.scatter(PC3_manual[mask, 0], PC3_manual[mask, 1], PC3_manual[mask, 2],
                c=colors[i], label=f'Class {int(cls)}', alpha=0.7)
ax1.set_title('Метод 1: Ручной PCA (NumPy) - 3 компоненты')
ax1.set_xlabel('PC1'); ax1.set_ylabel('PC2'); ax1.set_zlabel('PC3')
ax1.legend()

ax2 = fig.add_subplot(122, projection='3d')
for i, cls in enumerate(classes):
    mask = (y == cls)
    ax2.scatter(PC3_sklearn[mask, 0], PC3_sklearn[mask, 1], PC3_sklearn[mask, 2],
                c=colors[i], label=f'Class {int(cls)}', alpha=0.7)
ax2.set_title('Метод 2: Scikit-Learn PCA - 3 компоненты')
ax2.set_xlabel('PC1'); ax2.set_ylabel('PC2'); ax2.set_zlabel('PC3')
ax2.legend()

plt.tight_layout()
plt.show()

```

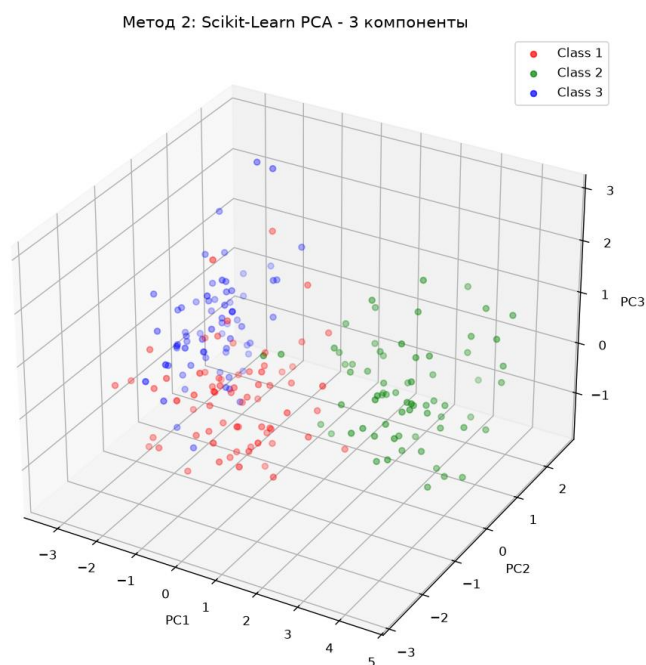
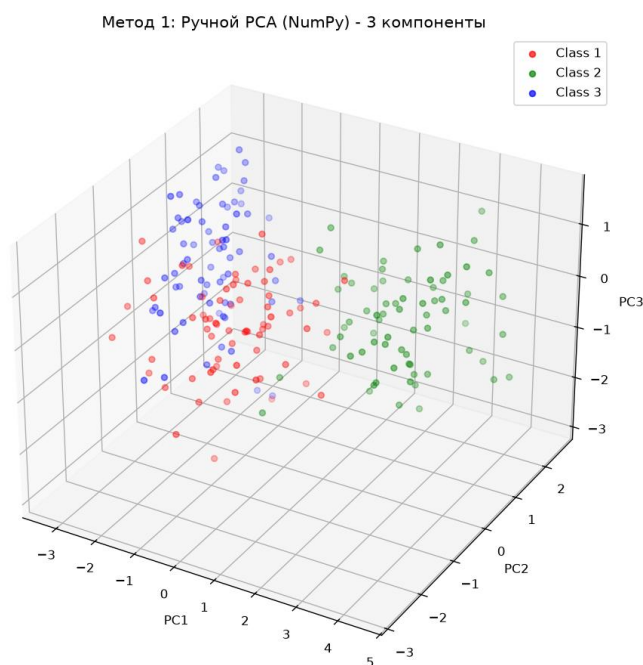
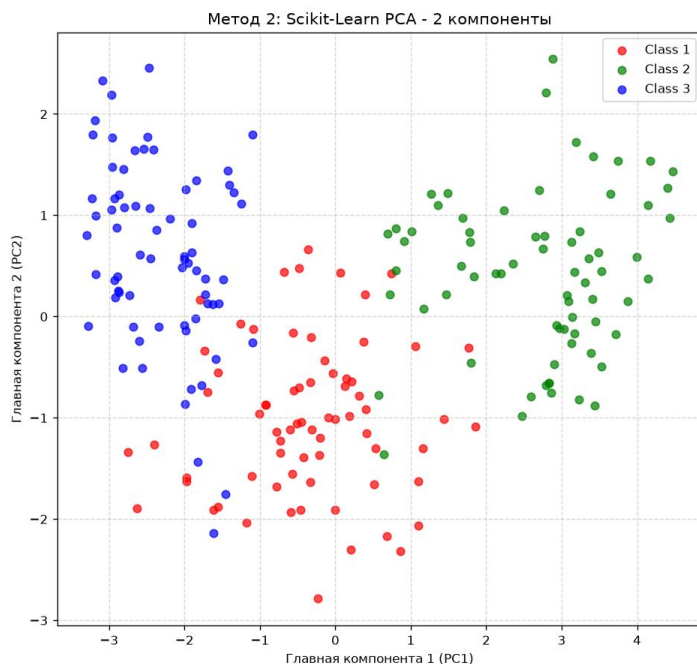
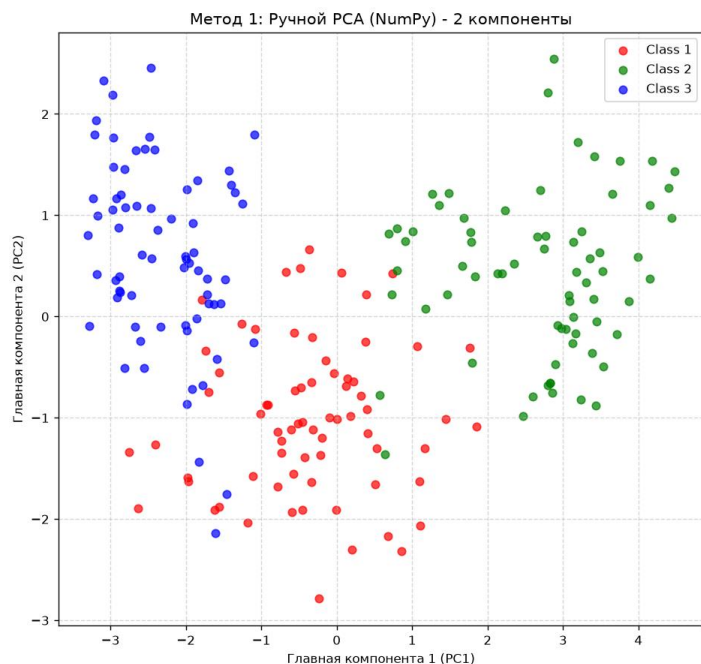
Результат программы:

```

C:\Users\Ксения\PycharmProjects\Labs\.venv\Scripts\python.exe
--- Расчет потерь информативности ---
Потери информативности при переходе к 2 компонентам: 11.02%
Потери информативности при переходе к 3 компонентам: 1.33%

Process finished with exit code 0

```



Выводы о проделанной работе:

1. *Корректность реализации:* результаты, полученные при ручной реализации алгоритма через `numpy.linalg.eig` и при использовании `sklearn.decomposition.PCA`, полностью совпадают (графики идентичны), что подтверждает правильность математической реализации метода.

2. *Качество разделения классов:* визуализация показывает, что три класса семян (Kama, Rosa, Canadian) имеют четкие различия и хорошо кластеризуются в пространстве главных компонент. Первая главная компонента (PC1) сильнее всего разделяет классы, что объясняется тем, что она в большей степени отвечает за общий размер зерна (площадь, периметр, длину).

3. *Потери информативности:*

- Переход к 2D-пространству сопровождается потерей всего 11.02% дисперсии, что является очень хорошим результатом. Этого достаточно для наблюдения явной кластерной структуры данных.
- Переход к 3D-пространству снижает потери до 1.33%, что означает сохранение почти всей исходной информации (98.67%).

4.Эффективность PCA: метод PCA доказал свою эффективность для визуализации многомерных данных. Даже при значительном сокращении размерности (с 7 признаков до 2-3) сохраняется большая часть информации о структуре данных, что позволяет наглядно анализировать кластеры и взаимосвязи между объектами.

5.Практическая значимость: полученные результаты показывают, что PCA может успешно применяться для:

- Визуализации многомерных данных
- Выделения признаков (feature extraction)
- Уменьшения размерности перед обучением моделей машинного обучения
- Анализа структуры данных и выявления кластеров.